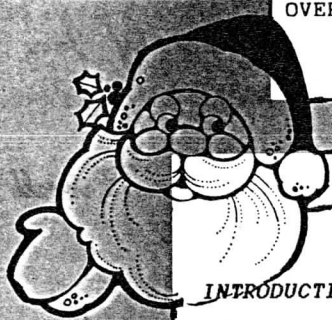
SVI & MSX

SPECTRAVIDEO

NEWSLETTER

REGISTERED BY AUSTRALIA POST PUBLICATION No. TBH 0917 CATEGORY "B"

ISSUE NO.	ANNUAL SUBSCRIPTION	DATE
3 - 3	AUSTRALIA ..... $20.00 OVERSEAS ..... $25.00 OVERSEAS AIRMAIL ... $30.00	DEC - 1985



CONTENTS

NEWSLETTER CORRESPONDENCE

S.A.U.G.,
P.O. BOX 191,
LAUNCESTON SOUTH,
TASMANIA, 7249.

(003) 442493

LIBRARY CORRESPONDENCE

S.A.U.G. LIBRARY,
1 CONRAD AVENUE,
GEORGE TOWN,
TASMANIA, 7253.

(003) 822919

UNDERSTANDING CP/M Pt-4

By S.W. McNamee

Well folks at long last here is the last installment in my series UNDERSTANDING CP/M. As I said in the first article this chapter will be concerned with the BIOS and its' peculiarities as installed on the SV-328.

First some generalities. All BIOS'S (BIOSES?, BIOI??, ?????) begin with a table of jumps with exactly 17 vectors in it. A vector by the way is just an address to jump to. These vectors point to routines in the BIOS that do all the low level I/O for the particular machine that it was written for and must be customised for that machine. I will not explain in detail what each routine does as this is well documented in Section 6.6 of the CP/M Operating System Manual. What I will do is take each jump in turn and discuss any special parts of it which the Spectravideo does a little differently to other machines. In 2.2 and 2.22 the BIOS jump table starts at E600H and is 51 bytes long (17 vectors) and appears thus for 2.22:

```
BOOT:     JP   EEEC
WBOOT:    JP   EF4C
CONST:    JP   E7A4
CONIN:    JP   E7AF
CONOUT:   JP   E7BA
LIST:     JP   E7D0
PUNCH:    JP   E7DB
READER:   JP   E7E6
HOME:     JP   EC78
SELDSK:   JP   EC5C
SETTRK:   JP   EC7A
SETSEC:   JP   EC7F
SETDMA:   JP   EC8E
READ:     JP   EC93
WRITE:    JP   ECD0
LISTST:   JP   E7C5
SECTRAN:  JP   EC58
```

The vectors are different for 2.2 due to its' shorter length but the principle is the same.

BOOT:

This is the routine that is jumped to immediately after the cold boot loader. I guess a brief word about the cold boot loader is in order. Most CP/M machines only have a very small ROM which is only responsible for reading in the first sector of the first track of a master disk in the A: drive. This sector contains code which switches out the ROM and then loads in the rest of the BIOS. The Spectravideo has a small routine in its' BASIC ROM that does this for us. If it finds that there is no disk in the drive (or no drive!) it jumps to BASIC.

Once the cold boot loader has loaded in the whole BIOS it jumps to the first vector in the jump table ... the BOOT routine. This section of code does things such as initialise any buffer areas, programme peripheral hardware (Such as RS-232 card or 80 Column card etc.) and print the sign on message. In the Spectravideo, calls are also made to the BASIC ROM to initialise the keyboard interrupt and the drivers for the 40 column display. The Spectravideo cold boot routine jumps to the warm boot routine when it is finished and it is this code that finishes loading in the rest of the system.

WBOOT:

This code is executed any time a Warm Boot is done. (Either by an executing program or by ^C from the keyboard.) It uses special sections of the BIOS to reload the BDOS and CPR and re-initialise the vectors at 0000H and 0005H. After it is finished the warm boot jumps to the CPR which then initialises the drive system, logs in the A: drive and prompts the user with the familiar 'A)'. A vector for this routine is stored at 0000H so that a warm boot will be done any time a program jumps to 0000H.

CONST:

This returns the status of the keyboard in A. The keyboard status routine in BASIC ROM at 3BH is actually called for this function. Since the BASIC ROM scans the keyboard on an interrupt driven basis the BIOS must keep this interrupt sequence going. Thus the BIOS switches in the BASIC ROM on every interrupt and calls its' interrupt handler.

CONIN:

Returns the next character from the keyboard buffer in A. The keyboard input routine in BASIC ROM at 3EH is actually called.

CONOUT:

Takes a byte in C and outputs it to the CONsole device. If the 80 column card is not being used this function calls the screen out vector in BASIC ROM at 18H.

LIST:

Takes a byte in C and outputs it to the LiST device.

PUNCH:

Takes a byte in C and outputs it to the PUNch device.

READER:

Returns the next byte from the ReaDeR device in A.

LISTST:

Returns the status of the LiST device in A.

The above routines share some common code in the Spectravideo versions and all implement the I/O byte concept. For a more involved discussion of this see page 138-139 of the CP/M manual. All these routines are of the form:

```
call    iobtst
dw      vect1
dw      vect2
dw      vect3
dw      vect4
```

The routine 'iobtst' tests the I/O byte and selects one of the four vectors following the call to it depending on the setting of the particular 2 bits in the I/O byte. It then branches to that vector.

HOME:
Sets the track counter to 0

SELDSK:
Takes a disk No. in C and selcts that disk. Returns the address of the DPH in HL.

SETTRK:
Sets the track counter to the value in C.

SETSEC:
Sets the sector counter to the value in C. Note that logical 128 byte sectors are used.

SETDMA:
Takes the address for the next 128 byte disk transfer in BC and sets the buffer.

READ:
Reads the 128 byte block from the previously selected disk, track and sector into the selected address.

WRITE:
Writes the 128 byte block from the selected address to the selected disk, track and sector.

SECTRAN:
Takes a sector number in BC (starting at 0) and returns a Physical sector number in HL. The spectravideo has all sector skewing physically placed on the disk at format time so no sector translation is needed. This routine simply increments the sector number by one and returns it in HL.

As well as these standard vectors the Spectravideo BIOS has a second jump table at E651H. These routines perform some extra duties for the system utilities and can be used by programers to their advantage.

E651 - Takes an address in HL and calls the BASIC ROM ... VERY useful.

E654 - This routine reads a block of PHYSICAL sectors from disk to a buffer. The call to this routine should be followed by a series of six bytes. It will check these bytes, read the selected block from disk and then return to the instruction immediately after the six bytes. In an assembly language program a call to this routine would look like this:

```
rdblck    equ  0e654h
          call rdblck
          dw   buffer
          db   sector
          db   track
          db   drive
          db   count
```

next:

'buffer' is the address to read the block to.
'sector' is the physical sector number to start reading from.
'track' is the track number
'drive' is the drive number (0 = drive A:).
'count' is the number of physical sectors to read. When the routine is finished it will return to the address labled 'next:'.
Note that track 0 is single density and has 18 128 byte sectors and all other tracks are double density and have 17 256 byte sectors. The routine allows for this and transfers an exact number of sectors whether they are single or double density. You should keep this in mind if you are reading from track 0 (as when accessing the system tracks).

E657 - This routine is exactly the same as the previous one except this one WRITES a block of memory to disk.

E65A - Calling this vector resets the disk timer.That is it keeps the drive motor on. It must be called every 10 seconds or so if you want the drive motor to stay on. Note that this routine is automatically called every time a sector is read from or written to disk. You only need to use this routine if you want the drive to stay on but do not want to access it for a while.

I will now talk a little bit about the 80 column card. This card uses a direct memory video RAM from F000H to F77FH, controlled by a 6545 CRT controller IC. The top left hand character is at F000 while the bottom right hand character is at F77F. The video RAM does not take up main memory however because it is bank switched. It is switched in when needed and then switched back out again. To switch in VRAM send 0FFH to port 58H and to switch it back out send 0 to the same port. The character set used is exactly the same as the colour display in BASIC. Ei The ASCII characters go from 0 - 5FH, the inverse ASCII is from 60H - BFH and the graphics characters start at C0H. The last characters in the set are the now familiar letters 'J.Suzuki'.

Three points are very important.

1.ALWAYS disable interrupts before switching in VRAM. You can enable them when you switch VRAM off. This is because the interrupt routine has a stack in VRAMs' address space, and also calls the BASIC ROM which also has stacks and vectors in the same address space.

2.NEVER switch in VRAM when your program is running in the area F000 - F800, and NEVER allow a program to CALL this area or JP to it.

3.NEVER locate your stack in this area when switching in VRAM.

If you ignore these rules your program will start running quite happily in VRAM with completely unpredictable results.

One more point concerning the 80 column card can be quite useful. There is a table in the BIOS used at cold boot to initialise the registers of the 6556 CRT controller. It is 16 bytes long, 1 byte for each of the first 16 registers which are the only ones used in the Spectravideo circuit configuration. In version 2.2 this table is at EB54H while in 2.22 it is at EC11H. Changing these values can allow you to customise the format of the 80 column display to a certain degree. For example although only 1920 bytes of VRAM are used there are actually 2048 bytes available. Changing the CRT register contents can allow you to display these hidden bytes. If you would like DATA sheets on the CRT controller chip send me a SAE and I will send you a copy.

My Address is:

S.W.McNamee
5/15 Stuckey Rd.
Clayfield 4011.

The last topic I will deal with is a way of modifying the CP/M system tracks of a disk. My preferred method is to use SYSGEN. Run SYSGEN as normal and when it asks for a destination drive press <ENTER>. A system disk will then be requested and you can reboot. Immediately after rebooting SAVE 51 CPM.COM (or some other appropriate file name. You will then have on disk a copy of the SYSGEN utility complete with a memory image of the operating system track. You can then run DDT and read in this file and modify it. When you have finished the modification just type G100 (from the DDT prompt) and the modified utility will start running. When it asks for the source drive hit <ENTER> to skip the read stage and then procede as normal specifying a destination drive. When finished a copy of the modified system tracks will be placed on the selected drive. The addresses of the various parts of the system in the memory image of the saved file are as follows:

BOOT LOADER	(128 bytes)	900H
CCP	(2048 bytes)	980H
BDOS	(3584 bytes)	1180H
BIOS	(2688 bytes)	1F80H

Although the cold boot loader only loads in 2688 bytes of BIOS there is actually room on the disk for 6528 bytes. To load in these sectors the cold boot loader sector needs to be changed and this is left as an exercise for the reader. All addresses in the BIOS are offset by C680H in the SYSGEN image. For example say you had version 2.22 and you wanted to modify the CRT register table at EC11. You would find this table at 2591H in the sysgen image and this is where you would make the change(2591H = EC11 - C680).

As noted earlier on the BIOS calls the BASIC ROM to perform some of its' functions. This means that you have to be careful in touching any RAM above F4FFH as this is BASICs' scratch pad area. However since CP/M only uses a small part of the BASIC ROM there are whole chunks of memory that are never touched and can be used as buffers etc. Their addresses and length follow:

F550 - F78F	575 bytes
F810 - F86F	95 bytes
F880 - F8DF	95 bytes
F900 - F97F	128 bytes
FB10 - FCDF	463 bytes

These are areas of RAM that I know to be safe from BASIC. There may be others but they will all be fairly small and not very useful. All RAM below F500 is safe from BASIC.

Well that just about wraps up this series of articles on CP/M. I hope you have enjoyed them and perhaps have gleaned a little bit of useful information from them. If any body would like to see some other topics discussed please write to the editor. If enough people show an interest in a particular subject I will endeavour to put 'fingers to keyboard' and churn out some words of wisdom???

P.S. The library has a file available which is a complete dissassemly of the Spectravideo BIOS. It can be modified and reassembled if desired, but is only partially documented. Both versions 2.2 and 2.22 are available.